

Лабораторная работа №2.

Хеширование и безопасное хранение паролей

Тема: Использование Argon2id для безопасного password storage в Express-приложении

Цель: Реализовать безопасное хеширование и проверку паролей через Argon2, интегрировать passwordService в регистрацию и вход пользователя и убедиться, что исходный password не сохраняется и не возвращается клиенту.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом `express-shop`. В проекте уже существуют User, Auth Service, endpoints регистрации и входа из предыдущей лабораторной работы.

Главное правило

Исходный password нельзя сохранять в PostgreSQL, выводить в логи или возвращать в HTTP-ответе. В базе должен находиться только passwordHash.

1. Проверьте пакет argon2

```
PowerShell  
npm install argon2
```

Пакет `argon2` предоставляет функции `hash()` и `verify()`. Для password hashing используется Argon2id.

2. Создайте отдельный passwordService

```
services/passwordService.js  
  
import * as argon2 from "argon2";  
  
export async function hashPassword(password) {  
  return argon2.hash(password);  
}  
  
export async function verifyPassword(passwordHash, password) {  
  return argon2.verify(passwordHash, password);  
}
```

Не генерируйте salt вручную. Библиотека создаёт уникальный salt автоматически и включает его в encoded hash.

3. Проведите эксперимент с одинаковым password

```
argon2-test.js  
  
import * as argon2 from "argon2";
```

```
const password = "Example123!";

const hash1 = await argon2.hash(password);
const hash2 = await argon2.hash(password);

console.log("Hash 1:", hash1);
console.log("Hash 2:", hash2);
console.log("Equal:", hash1 === hash2);

console.log(
  "Verify hash1:",
  await argon2.verify(hash1, password)
);

console.log(
  "Wrong password:",
  await argon2.verify(hash1, "wrong-password")
);
```

PowerShell

```
node argon2-test.js
```

- `hash1` и `hash2` должны отличаться.
- `hash1 === hash2` должен вернуть `false`.
- Проверка правильного `password` должна вернуть `true`.
- Проверка неправильного `password` должна вернуть `false`.

Почему hashes разные

Для каждого вызова `argon2.hash()` используется новый случайный `salt`. Поэтому одинаковый `password` не обязан давать одинаковую строку `hash`.

4. Рассмотрите структуру encoded hash

Выведите значение `hash1`. В начале строки должно быть обозначение алгоритма `Argon2id` и параметры вычисления.

Схематичный пример

```
$argon2id$v=19$m=...,t=...,p=...$<salt>$<hash>
```

- `argon2id` — вариант алгоритма.
- `m` — memory cost.
- `t` — time cost.
- `p` — parallelism.
- После параметров находятся `salt` и итоговый `hash`.

5. Проверьте модель User

```
prisma/schema.prisma
```

```
model User {
```

```

id          Int          @id @default(autoincrement())
name        String
email       String       @unique
passwordHash String      @map("password_hash")
role        String       @default("user")
orders      Order[]

@@map("users")
}

```

Не создавайте поле password

В модели User не должно быть отдельного поля для хранения исходного password.

6. Обновите registerUser()

```

services/authService.js

import { prisma } from "../config/prisma.js";
import {
  hashPassword,
  verifyPassword
} from "../passwordService.js";

export async function registerUser({ name, email, password }) {
  const existing = await prisma.user.findUnique({
    where: { email }
  });

  if (existing) {
    throw new Error("Email already registered");
  }

  const passwordHash = await hashPassword(password);

  return prisma.user.create({
    data: {
      name,
      email,
      passwordHash
    },
    select: {
      id: true,
      name: true,
      email: true,
      role: true
    }
  });
}

```

`password` используется только для вызова `hashPassword()`. В Prisma передаётся уже `passwordHash`.

7. Проверьте регистрацию

```
POST /api/auth/register
```

```
POST /api/auth/register
Content-Type: application/json
```

```
{
  "name": "Anna",
  "email": "anna@example.com",
  "password": "Example123!"
}
```

Ожидаемый ответ

```
{
  "user": {
    "id": 2,
    "name": "Anna",
    "email": "anna@example.com",
    "role": "user"
  }
}
```

Фактическое значение `id` зависит от состояния базы данных. Поля `password` и `passwordHash` в ответе отсутствуют.

8. Проверьте запись в PostgreSQL

Проверка users

```
SELECT
  id,
  name,
  email,
  password_hash,
  role
FROM users
WHERE email = 'anna@example.com';
```

В колонке `password_hash` должна находиться encoded Argon2id string. Исходного `Example123!` в таблице быть не должно.

9. Обновите `authenticateUser()`

services/authService.js

```
export async function authenticateUser(email, password) {
  const user = await prisma.user.findUnique({
    where: { email }
  });

  if (!user) {
    return null;
  }

  const valid = await verifyPassword(
    user.passwordHash,
    password
  );
}
```

```
if (!valid) {
  return null;
}

return {
  id: user.id,
  name: user.name,
  email: user.email,
  role: user.role
};
}
```

Не вызывайте `hashPassword(password)` во время `login` для последующего сравнения строк. Из-за нового `salt` новый `hash` будет отличаться. Для проверки используйте только `verifyPassword()`.

10. Проверьте правильный password

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "anna@example.com",
  "password": "Example123!"
}
```

Запрос должен завершиться успешно и вернуть безопасные данные `User`.

11. Проверьте неправильный password

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "anna@example.com",
  "password": "wrong-password"
}
```

Ожидаемый ответ

```
{
  "error": "Invalid email or password"
}
```

Ожидаемый HTTP status — 401.

12. Проверьте неизвестный email

```
POST /api/auth/login
Content-Type: application/json

{
```

```
"email": "missing@example.com",  
"password": "Example123!"  
}
```

```
{  
  "error": "Invalid email or password"  
}
```

Одинаковая ошибка

Неизвестный email и неправильный password должны давать одинаковое внешнее сообщение, чтобы login endpoint не раскрывал наличие учётной записи.

13. Проверьте отсутствие passwordHash в API

1. Выполните регистрацию.
2. Выполните успешный login.
3. Проверьте каждый JSON-ответ.
4. Убедитесь, что поле passwordHash отсутствует.
5. Проверьте Controller и Service на случайный `res.json(user)` с полным объектом Prisma.

14. Измерьте примерное время hashing

hash-benchmark.js

```
import * as argon2 from "argon2";  
  
const password = "Example123!";  
  
console.time("argon2.hash");  
const hash = await argon2.hash(password);  
console.timeEnd("argon2.hash");  
  
console.time("argon2.verify");  
await argon2.verify(hash, password);  
console.timeEnd("argon2.verify");
```

PowerShell

```
node hash-benchmark.js
```

Зафиксируйте полученное время. Password hashing намеренно выполняется медленнее обычных быстрых hash-функций.

Не оптимизируйте до максимальной скорости

Для password hashing высокая скорость не является целью. Алгоритм должен быть достаточно дорогим для массового brute force, но приемлемым для обычного login.

15. Самостоятельное задание

- Создайте двух пользователей с одинаковым password.
- Сравните их password_hash в PostgreSQL.
- Объясните, почему hashes различаются.
- Проверьте оба аккаунта через login.
- Создайте эксперимент с двумя вызовами `argon2.hash()` для одной строки.
- Докажите, что обычное сравнение hashes через `===` не подходит.
- Покажите, что `argon2.verify()` корректно проверяет password.
- Убедитесь, что passwordHash отсутствует во всех HTTP-ответах.

Требования к результату

- Для password hashing используется пакет `argon2`.
- Криптографическая логика находится в `passwordService.js`.
- Регистрация использует `hashPassword()`.
- Login использует `verifyPassword()`.
- В PostgreSQL хранится passwordHash, а не исходный password.
- Два одинаковых passwords создают разные encoded hashes.
- Правильный password успешно проходит verify.
- Неправильный password не проходит verify.
- passwordHash отсутствует в HTTP-ответах.
- Студент может объяснить назначение salt и memory-hard hashing.

Критерии оценивания

№	Критерий
1	Пакет <code>argon2</code> установлен
2	Создан <code>passwordService.js</code>
3	<code>hashPassword()</code> использует <code>argon2.hash()</code>
4	<code>verifyPassword()</code> использует <code>argon2.verify()</code>
5	User хранит passwordHash
6	Исходный password не сохраняется
7	Регистрация создаёт Argon2id hash
8	Два hash одинакового password различаются
9	Правильный password проходит verify
10	Неправильный password не проходит verify
11	Login возвращает 401 при неверных credentials
12	passwordHash отсутствует в register response
13	passwordHash отсутствует в login response
14	Студент объясняет роль salt

№	Критерий
15	Студент объясняет, почему SHA-256 не подходит для password storage